

# Problem Solving and testing using Java

## Week-1 Task Questions

Name: C.HIMATEJA

VTU:28828

## SESSION-1: Introduction to Functional Programming Concepts

### 1. Java Lambda Expressions -

<https://www.hackerrank.com/challenges/java-lambda-expressions/problem>

#### PROBLEM STATEMENT:

This Java 8 challenge tests your knowledge of [Lambda expressions!](#)

Write the following methods that *return a lambda expression* performing a specified action:

1. PerformOperation isOdd(): The lambda expression must return TRUE if a number is odd or if it is even.
2. PerformOperation isPrime(): The lambda expression must return TRUE if a number is prime or if it is composite.
3. PerformOperation isPalindrome(): The lambda expression must return FALSE if a number is a palindrome or if it is not.

#### Input Format

Input is handled for you by the locked stub code in your editor.

#### Output Format

The locked stub code in your editor will print lines of output.

#### Sample Input

The first line contains an integer, T (the number of test cases).

The T subsequent lines each describe a test case in the form of 2-space-separated integers:

The first integer specifies the condition to check for (1 for Odd/Even, 2 for Prime, or 3 for Palindrome). The second integer denotes the number to be checked.

5

1 4

2 5

3 898

1 3

2 12

#### Sample Output

EVEN

PRIME

PALINDROME

ODD

COMPOSITE

#### PROGRAM:

```
import java.io.*;
import java.util.*;
interface PerformOperation {
```

```

boolean check(int a);
}
class MyMath {
    public static boolean checker(PerformOperation p, int num) {
        return p.check(num);
    }

    public PerformOperation isOdd() {
        return (n) -> n % 2 != 0;
    }

    // 2. Check Prime
    public PerformOperation isPrime() {
        return (n) -> {
            if (n < 2)
                return false;

            for (int i = 2; i <= Math.sqrt(n); i++) {
                if (n % i == 0)
                    return false;
            }
            return true;
        };
    }

    // 3. Check Palindrome
    public PerformOperation isPalindrome() {
        return (n) -> {
            int original = n;
            int reverse = 0;

            while (n > 0) {
                int digit = n % 10;
                reverse = reverse * 10 + digit;
                n = n / 10;
            }
            return original == reverse;
        };
    }
}

public class Solution {
    public static void main(String[] args) throws IOException {
        MyMath ob = new MyMath();
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    }
}

```

```

int T = Integer.parseInt(br.readLine());
PerformOperation op;
boolean ret = false;
String ans = null;
while (T--> 0) {
    String s = br.readLine().trim();
    StringTokenizer st = new StringTokenizer(s);
    int ch = Integer.parseInt(st.nextToken());
    int num = Integer.parseInt(st.nextToken());
    if (ch == 1) {
        op = ob.isOdd();
        ret = ob.checker(op, num);
        ans = (ret) ? "ODD" : "EVEN";
    }
    else if (ch == 2) {
        op = ob.isPrime();
        ret = ob.checker(op, num);
        ans = (ret) ? "PRIME" : "COMPOSITE";
    }
    else if (ch == 3) {
        op = ob.isPalindrome();
        ret = ob.checker(op, num);
        ans = (ret) ? "PALINDROME" : "NOT PALINDROME";
    }
    System.out.println(ans);
}
}
}

```

## OUTPUT:

**Congratulations!**

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

```

1 ✓
2 1 4
3 2 5
4 3 898
5 1 3
6 2 12

```

Your Output (stdout)

```

1 EVEN
2 PRIME
3 PALINDROME
4 ODD
5 COMPOSITE

```

## 1-2.Programming Concepts : Task 1 in syllabus

### Problem Description

A company stores employee salaries in a list. Increase each salary by 10% and print the updated salaries using functional transformation techniques.

Input Format

- First line contains integer N.
- Second line contains N space-separated integers.

Output Format

Print updated salaries.

Constraints

- $1 \leq N \leq 10^5$

Sample Input

5

1000 2000 3000 4000 5000

Sample Output

1100 2200 3300 4400 5500

### Program:

```
package TIMES_DATES;
import java.util.*;
import java.util.stream.Collectors;
public class sale_management {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        List<Integer> salaries = new ArrayList<>();
        for (int i = 0; i < N; i++) {
            salaries.add(sc.nextInt());
        } // Increase each salary by 10%
        List<Integer> updated = salaries.stream()
            .map(salary -> salary + salary / 10)
            .collect(Collectors.toList());
        // Print updated salaries
        for (int s : updated) {
            System.out.print(s + " ");
        }
        sc.close();
    }
}
```

## OUTPUT:

```
<terminated> sale_management [Java Application] C:\Program Files\Java\jdk-25\
5
1000 2000 3000 4000 5000
1100 2200 3300 4400 5500
```

## Session 2: Lambda Expressions

### 1. Java Sort – <https://www.hackerrank.com/challenges/java-sort/problem>

#### PROBLEM STATEMENT:

You are given a list of student information: ID, FirstName, and CGPA. Your task is to rearrange them according to their CGPA in decreasing order. If two student have the same CGPA, then arrange them according to their first name in alphabetical order. If those two students also have the same first name, then order them according to their ID. No two students have the same ID.

**Hint:** You can use comparators to sort a list of objects. See the [oracle docs](#) to learn about comparators.

#### Input Format

The first line of input contains an integer , representing the total number of students. The next lines contains a list of student information in the following structure:

ID Name CGPA

#### Constraints:

The name contains only lowercase English letters. The contains only integer numbers without leading zeros. The *CGPA* will contain, at most, 2 digits after the decimal point.

#### Output Format

After rearranging the students according to the above rules, print the first name of each student on a separate line.

#### Sample Input

```
5
33 Rumpa 3.68
85 Ashis 3.85
56 Samiha 3.75
19 Samara 3.75
22 Fahim 3.76
```

#### Sample Output

```
Ashis
Fahim
Samara
Samiha
```

Rumpa

### Program:

```
import java.util.*;
class Student{
private int id;
private String fname;
private double cgpa;
public Student(int id, String fname, double cgpa) {
    super();
    this.id = id;
    this.fname = fname;
    this.cgpa = cgpa;
}
public int getId() {
    return id;
}
public String getFname() {
    return fname;
}
public double getCgpa() {
    return cgpa;
}
}
```

//Complete the code

```
public class Solution
{
public static void main(String[] args){
    Scanner in = new Scanner(System.in);
    int testCases = Integer.parseInt(in.nextLine());

    List<Student> studentList = new ArrayList<Student>();
    while(testCases>0){
        int id = in.nextInt();
        String fname = in.next();
        double cgpa = in.nextDouble();

        Student st = new Student(id, fname, cgpa);
        studentList.add(st);

        testCases--;
    }
}
```

//Complete the code

```
Collections.sort(studentList, (s1, s2) -> {
    if (Double.compare(s2.getCgpa(), s1.getCgpa()) != 0) {
```

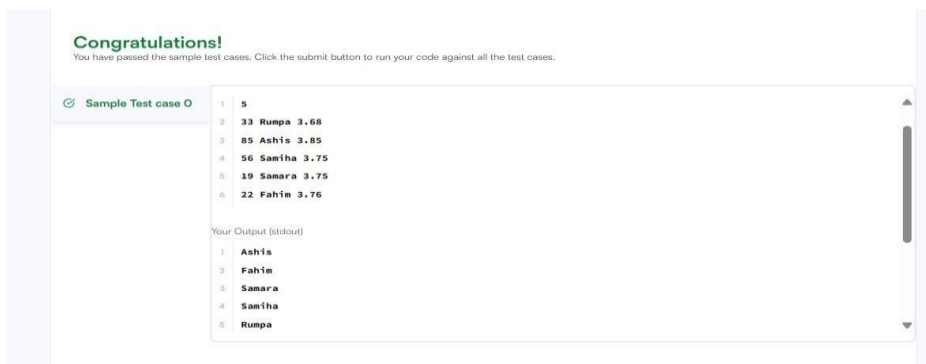
```

        return Double.compare(s2.getCgpa(), s1.getCgpa());
    }
    else if (!s1.getFname().equals(s2.getFname())) {
        return s1.getFname().compareTo(s2.getFname());
    }
    else {
        return s1.getId() - s2.getId();
    }
});

for(Student st: studentList){
    System.out.println(st.getFname());
}
}
}

```

OUTPUT:



## 2. Sort Array By Parity

<https://leetcode.com/problems/sort-array-by-parity/>

### PROBLEM STATEMENT:

Given an integer array `nums`, move all the even integers at the beginning of the array followed by all the odd integers.

Return *any array that satisfies this condition*.

Example 1:

Input: `nums = [3,1,2,4]`

Output: `[2,4,3,1]`

Explanation: The outputs `[4,2,3,1]`, `[2,4,1,3]`, and `[4,2,1,3]` would also be accepted.

Example 2:

Input: `nums = [0]`

Output: `[0]`

Constraints:

- $1 \leq \text{nums.length} \leq 5000$



- $0 \leq \text{nums}[i] \leq 5000$

## PROGRAM:

```
class Solution {
    public int[] sortByParity(int[] nums) {
        int left = 0;
        int right = nums.length - 1;
        while (left < right) {
            while (left < right && nums[left] % 2 == 0) {
                left++;
            }
            while (left < right && nums[right] % 2 != 0) {
                right--;
            }

            int temp = nums[left];
            nums[left] = nums[right];
            nums[right] = temp;

            left++;
            right--;
        }
        return nums;
    }
}
```

## OUTPUT:

The screenshot displays the LeetCode interface for problem 905, "Sort Array By Parity". The problem description asks to move all even integers to the beginning of the array followed by all odd integers. The provided Java solution uses a two-pointer approach: a left pointer moves right until it finds an odd number, and a right pointer moves left until it finds an even number, then they swap. The test results show two cases passed successfully.

**Problem Description:** 905. Sort Array By Parity

Given an integer array `nums`, move all the even integers at the beginning of the array followed by all the odd integers. Return *any* array that satisfies this condition.

**Example 1:**  
**Input:** `nums = [3,1,2,4]`  
**Output:** `[2,4,3,1]`  
**Explanation:** The outputs `[4,2,3,1]`, `[2,4,1,3]`, and `[4,2,1,3]` would also be accepted.

**Example 2:**  
**Input:** `nums = [0]`  
**Output:** `[0]`

**Constraints:**

- $1 \leq \text{nums.length} \leq 5000$
- $0 \leq \text{nums}[i] \leq 5000$

**Code Editor:**

```
Java
14
15
16
17
18
19
20
21
    right--;
    }
    int temp = nums[left];
    nums[left] = nums[right];
    nums[right] = temp;
    left++;
    }
```

**Testcase / Test Result:**

**Accepted** Runtime: 0 ms

**Case 1** **Case 2**

**Input:** `nums = [3,1,2,4]`

**Output:** `[4,2,1,3]`

**Expected:** `[2,4,3,1]`

## SESSION-3: Anonymous Functions & Functional Interfaces

### 1.Sorting: Comparator –

<https://www.hackerrank.com/challenges/ctci-comparator-sorting/problem>

### PROBLEM STATEMENT:

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array. The *Player* class is provided in the editor below. It has two fields:

1. name: a string.
2. score: an integer.

Given an array of *n* *Player* objects, write a comparator that sorts them in order of decreasing score. If 2 or more players have the same score, sort those players alphabetically ascending by name. To do this, you must create a *Checker* class that implements the *Comparator* interface, then write an *int compare(Player a, Player b)* method implementing the [Comparator.compare\(T o1, T o2\)](#) method. In short, when sorting in ascending order, a comparator function returns -1 if *a* < *b*, 0 if *a* = *b*, and 1 if *a* > *b*.

Declare a *Checker* class that implements the *comparator* method as described. It should sort first descending by score, then ascending by name. The code stub reads the input, creates a list of *Player* objects, uses your method to sort the data, and prints it out properly.

Example

```
n = 3, data = [[Smith, 20], [Jones, 15], [Jones, 20]]
```

```
dataSorted = [[Jones, 20], [Smith, 20], [Jones, 15]]
```

Sort the list as data sorted. Sort first descending by score, then ascending by name.

Input Format

The first line contains an integer, *n*, the number of players.

Each of the next *n* lines contains a player's name, a string, and an integer.

Constraints

- $0 \leq \text{score} \leq 100$
- Two or more players can have the same name.
- Player names consist of lowercase English alphabetic letters.

Output Format

You are not responsible for printing any output to stdout. Locked stub code in *Solution* will instantiate a *Checker* object, use it to sort the *Player* array, and print each sorted element.

Sample Input

```
5
amy 100
david 100
heraldo 50
aakansha 75
aleksa 150
```

Sample Output

```
aleksa 150
```

amy 100  
david 100  
aakansha 75  
heraldo 50

## PROGRAM:

```
import java.util.*;

class Player {
    String name;
    int score;
    Player(String name, int score) {
        this.name = name;
        this.score = score;
    }
}

class Checker implements Comparator<Player> {
    // complete this method
    public int compare(Player a, Player b) {
        if (a.score != b.score) {
            return Integer.compare(b.score, a.score);
        }

        return a.name.compareTo(b.name);
    }
}

public class Solution {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();

        Player[] player = new Player[n];
        Checker checker = new Checker();

        for(int i = 0; i < n; i++){
            player[i] = new Player(scan.next(), scan.nextInt());
        }
        scan.close();

        Arrays.sort(player, checker);
        for(int i = 0; i < player.length; i++){
            System.out.printf("%s %s\n", player[i].name, player[i].score);
        }
    }
}
```

## OUTPUT:

**Congratulations!**  
You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0 ✓

Sample Test case 1 ✓

Sample Test case 2 ✓

Input (stdin)

```
1 5
2 amy 100
3 david 100
4 heraldo 50
5 aakansha 75
6 aleksa 150
```

Download

Your Output (stdout)

```
1 aleksa 150
2 amy 100
3 david 100
```

## 2. Java Comparator –

<https://www.hackerrank.com/challenges/java-comparator/problem>

### PROBLEM STATEMENT:

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

The *Player* class is provided for you in your editor. It has 2 fields: a name String and a score integer.

Given an array of *n* *Player* objects, write a comparator that sorts them in order of decreasing score; if 2 or more players have the same score, sort those players alphabetically by name. To do this, you must create a *Checker* class that implements the *Comparator* interface, then write an *int compare(Player a, Player b)* method implementing the [Comparator.compare\(T o1, T o2\)](#) method.

Input Format

Input from stdin is handled by the locked stub code in the *Solution* class.

The first line contains an integer, *n*, denoting the number of players.

Each of the subsequent lines contains a player's name and score, respectively.

Constraints

- $0 \leq \text{score} \leq 100$
- players can have the same name.
- Player names consist of lowercase English letters.

### Output Format

You are not responsible for printing any output to stdout. The locked stub code in *Solution* will create a *Checker* object, use it to sort the *Player* array, and print each sorted element.

### Sample Input

```
5
amy 100
david 100
heraldo 50
aakansha 75
aleksa 150
```

### Sample Output

```
aleksa 150
amy 100
david 100
aakansha 75
heraldo 50
```

## PROGRAM:

```
class Checker implements Comparator<Player> {
    @Override
    public int compare(Player a, Player b) {
        if (a.score != b.score) {
            return b.score - a.score;
        }
        return a.name.compareTo(b.name);
    }
}

class Player{
    String name;
    int score;
    Player(String name, int score){
        this.name = name;
        this.score = score;
    }
}

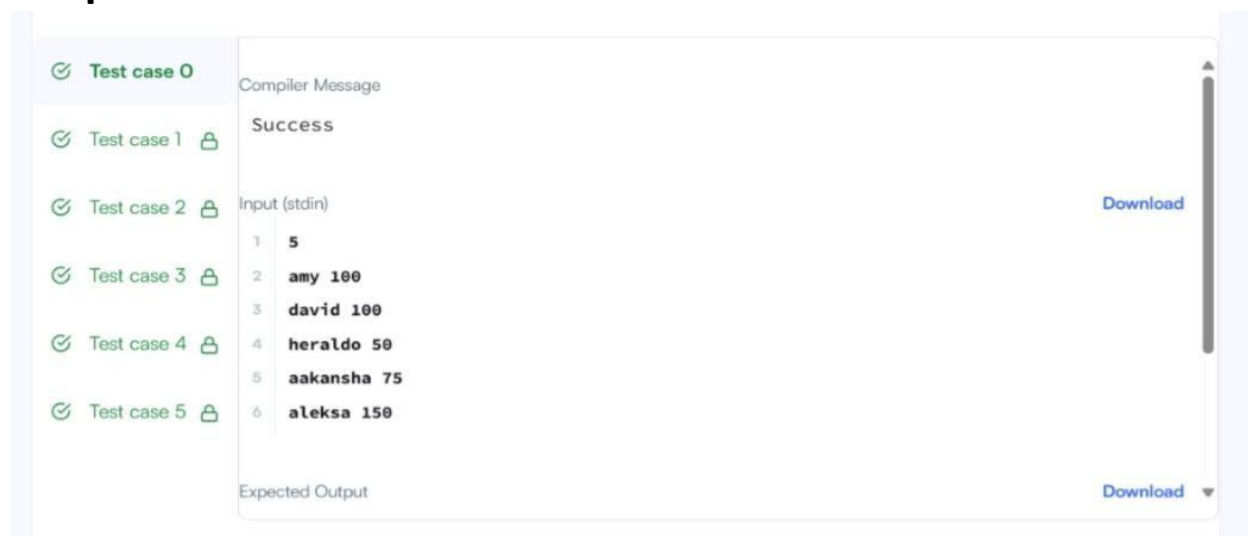
class Solution {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();
        Player[] player = new Player[n];
        Checker checker = new Checker();
        for(int i = 0; i < n; i++){
```

```

        player[i] = new Player(scan.next(), scan.nextInt());
    }
    scan.close();
    Arrays.sort(player, checker);
    for(int i = 0; i < player.length; i++){
        System.out.printf("%s %s\n", player[i].name, player[i].score);
    }
}
}

```

## Output:



## SESSION-4: Higher-Order Functions

### 2. Richest Customer Wealth –

<https://leetcode.com/problems/richest-customer-wealth/>

#### PROBLEM STATEMENT:

You are given an  $m \times n$  integer grid `accounts` where `accounts[i][j]` is the amount of money the  $i^{\text{th}}$  customer has in the  $j^{\text{th}}$  bank. Return *the **wealth** that the richest customer has*.

A customer's **wealth** is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum **wealth**.

#### Example 1:

**Input:** `accounts = [[1,2,3],[3,2,1]]`

**Output:** 6

#### Explanation:

1st customer has `wealth = 1 + 2 + 3 = 6`

2nd customer has `wealth = 3 + 2 + 1 = 6`

Both customers are considered the richest with a wealth of 6 each, so return 6.

### Example 2:

**Input:** accounts = [[1,5],[7,3],[3,5]]

**Output:** 10

**Explanation:**

1st customer has wealth = 6

2nd customer has wealth = 10

3rd customer has wealth = 8

The 2nd customer is the richest with a wealth of 10.

### Example 3:

**Input:** accounts = [[2,8,7],[7,1,3],[1,9,5]]

**Output:** 17

**Constraints:**

- $m == \text{accounts.length}$
- $n == \text{accounts}[i].\text{length}$
- $1 \leq m, n \leq 50$
- $1 \leq \text{accounts}[i][j] \leq 100$

## PROGRAM:

```
class Solution {
    public int maximumWealth(int[][] accounts) {
        int max=0;
        for(int person=0;person<accounts.length;person++){
            int sum=0;
            for(int account=0;account<accounts[person].length;account++){
                sum+=accounts[person][account];
            }
            if(sum>max){
                max=sum;
            }
        }
        return max;
    }
}
```

## OUTPUT:

The screenshot shows a test result interface with a tab labeled 'Test Result'. Below the tab, it says 'Accepted' in green and 'Runtime: 0 ms'. There are three test cases listed: 'Case 1', 'Case 2', and 'Case 3', each with a green checkmark. Below the test cases, there is an 'Input' section with the text 'accounts = [[1,2,3],[3,2,1]]'. Below the input, there is an 'Output' section with the text '6'. Below the output, there is an 'Expected' section with the text '6'.

